

SVM

Rodrigo Mansilla, Javier Chen

2025-02-25

Exploración de Datos

Table 1: Número de valores NA por columna

Columna	Cantidad de NA
MasVnrType	4
MasVnrArea	4
BsmtQual	26
BsmtCond	26
GarageYrBlt	57
GarageQual	57
GarageCond	57

Table 2: Umbrales para categorizar PriceCat

Cuartil	SalePrice
25% (Eco/Inter)	-1
75% (Inter/Caras)	1

Table 3: Distribución de categorías de precio

PriceCat	Train	Test
Economicas	256	109
Intermedias	509	219
Caras	259	108

Table 4: Dimensiones de los datasets con PriceCat

Dataset	Filas	Columnas
train_cat	1024	104
test_cat	436	104

- Focos de datos faltantes: La mayoría de los NA se concentran en atributos de sótano y garaje (**BsmtQual**, **BsmtCond**, **GarageYrBlt**, **GarageQual**, **GarageCond**), lo que sugiere revisar si imputarlos con la mediana o tratarlos como “sin sótano/garaje” para no perder información clave.

- Umbrales de PriceCat: Se fijaron en los cuartiles 25 % (`SalePrice < -1`) y 75 % (`SalePrice > 1`), garantizando cortes claros entre “Económicas”, “Intermedias” y “Caras”.
 - Balance de clases:
 - "Intermedias" es la más numerosa (~50 % del total).
 - "Económicas" y "Caras" están pareadas en tamaño (~25 % cada una), aunque "Intermedias" podría dominar.
 - Consistencia train/test: La proporción de cada categoría en entrenamiento (256/509/259) y prueba (109/219/108) preserva la distribución original, favoreciendo evaluaciones comparables.
 - Volumen de datos:
 - Train_cat: 1 024 filas × 104 columnas
 - Test_cat: 436 filas × 104 columnas
- Suficiente para entrenar modelos SVM sin riesgo excesivo de varianza por escasez de datos.
- Implicación para modelado: Dado el tamaño y la ligera clase mayoritaria en “Intermedias”, conviene monitorizar métricas más allá del accuracy (e.g. F1-score por clase) y considerar estratificación o ponderación en el entrenamiento.

Entrenamiento del modelo SVM

Table 5: Comparación de SVM: Train vs Test Accuracy

Modelo	Kernel	Grado	Costo	Gamma	Acc. Train	Acc. Test
svm_lin_C0.1	linear	NA	0.1	NA	0.958	0.812
svm_lin_C1	linear	NA	1.0	NA	0.982	0.803
svm_lin_C10	linear	NA	10.0	NA	0.991	0.800
svm_poly_deg2_C1_g0.01	polynomial	2	1.0	0.010	0.973	0.805
svm_poly_deg3_C1_g0.01	polynomial	3	1.0	0.010	0.968	0.791
svm_poly_deg2_C1_g0.1	polynomial	2	1.0	0.100	1.000	0.787
svm_poly_deg3_C1_g0.1	polynomial	3	1.0	0.100	1.000	0.775
svm_poly_deg2_C5_g0.01	polynomial	2	5.0	0.010	0.995	0.800
svm_poly_deg3_C5_g0.01	polynomial	3	5.0	0.010	0.997	0.784
svm_poly_deg2_C5_g0.1	polynomial	2	5.0	0.100	1.000	0.787
svm_poly_deg3_C5_g0.1	polynomial	3	5.0	0.100	1.000	0.775
svm_rad_C1_g0.001	radial	NA	1.0	0.001	0.887	0.812
svm_rad_C1_g0.01	radial	NA	1.0	0.010	0.973	0.814
svm_rad_C1_g0.1	radial	NA	1.0	0.100	1.000	0.507
svm_rad_C10_g0.001	radial	NA	10.0	0.001	0.951	0.853
svm_rad_C10_g0.01	radial	NA	10.0	0.010	1.000	0.789
svm_rad_C10_g0.1	radial	NA	10.0	0.100	1.000	0.509

Table 6: Resultados de tuning kernel radial

Gamma	Cost	Error	SD
0.125	0.5	0.498	0.069
0.250	0.5	0.498	0.069
0.500	0.5	0.498	0.069
0.125	1.0	0.497	0.069
0.250	1.0	0.498	0.069
0.500	1.0	0.498	0.069
0.125	2.0	0.493	0.066
0.250	2.0	0.498	0.069
0.500	2.0	0.498	0.069
0.125	4.0	0.493	0.066
0.250	4.0	0.498	0.069
0.500	4.0	0.498	0.069

Table 7: Mejor modelo radial

Mejor_Kernel	Costo	Gamma
radial	2	0.125

- Lineal vs. polinomial vs. radial
 - Lineal ($C=0.1-10$): buena precisión de prueba ($\sim 0.80-0.81$) con brecha Train-Test de $\sim 14-19$ puntos, sugiere algo de sobreajuste al subir C .
 - Polinomial (grado 2-3): al aumentar C o γ alcanza 100 % en entrenamiento pero cae a <0.79 en prueba, señal clara de overfitting.
 - Radial ($C=1-10$, $\gamma=0.001-0.1$): el mejor balance lo da $C=10$, $\gamma=0.001$ (Train 95.1 %, Test 85.3 %, Δ 9.8 pp), reduciendo el sobreajuste respecto a lineal/polinomial.
- Efecto de los hiperparámetros (Table 6-7)
- Grid search CV identificó como óptimos para kernel radial $C=2$ y $\gamma=0.125$, con error medio 0.493 (SD 0.069)
- Brecha Train-Test
 - La menor diferencia (~ 9.8 pp) se ve en radial $C=10$, $\gamma=0.001$; lineal y polinomial muestran diferencias de ~ 22 pp), confirmando que el kernel radial generaliza mejor.
- Selección del mejor modelo
 - Basado en Test Accuracy y brecha moderada, el SVM radial es el ganador, y el ajuste fino ($C=2$, $\gamma=0.125$)

Predicciones

Table 8: Accuracy de SVM en test_final

Modelo	Accuracy
--------	----------

svm_rad_C10_g0.001	0.8532110
svm_rad_C1_g0.01	0.8142202
svm_lin_C0.1	0.8119266
svm_rad_C1_g0.001	0.8119266
svm_poly_deg2_C1_g0.01	0.8050459
svm_lin_C1	0.8027523
svm_lin_C10	0.8004587
svm_poly_deg2_C5_g0.01	0.8004587
svm_poly_deg3_C1_g0.01	0.7912844
svm_rad_C10_g0.01	0.7889908
svm_poly_deg2_C1_g0.1	0.7866972
svm_poly_deg2_C5_g0.1	0.7866972
svm_poly_deg3_C5_g0.01	0.7844037
svm_poly_deg3_C1_g0.1	0.7752294
svm_poly_deg3_C5_g0.1	0.7752294
svm_rad_C10_g0.1	0.5091743
svm_rad_C1_g0.1	0.5068807
svm_best_radial	0.5045872

- svm_lin_Cx
 - lin: kernel lineal.
 - C=x: parámetro de penalización (regularización) igual a x.
- svm_poly_degD_Cx_gY
 - poly: kernel polinomial.
 - degD: grado del polinomio D.
 - C=x: coste de error x.
 - g=Y: gamma (influye en el "alcance" de cada punto de soporte) igual a Y.
- svm_rad_Cx_gY
 - rad: kernel radial (RBF).
 - C=x: coste de penalización x.
 - g=Y: gamma de la función RBF igual a Y.
- svm_best_radial
 - Modelo radial ajustado automáticamente con los hiperparámetros que dieron menor error en validación.

Insights

- El mejor modelo es svm_rad_C10_g0.001 con 85.32 % de accuracy en test, claramente por encima del resto.

- Le siguen de cerca svm_rad_C1_g0.01 (81.42 %) y svm_lin_C0.1 (81.19 %), lo que muestra que un kernel lineal bien regularizado compite con versiones radiales de bajo γ .
- Los polinomiales alcanzan como máximo ~80.5 % (grado 2, C=1, $\gamma=0.01$), indicando que su complejidad extra no se traduce en mejor performance.
- Las configuraciones con $\gamma=0.1$ (tanto lineal como radial) colapsan a ~50 %, un claro síntoma de overfitting que las hace inútiles para clasificación de tres clases.
- La coherencia entre los resultados de CV (Table 6–7) y estas accuracies confirma que los mejores rangos de γ están en [0.001–0.01].
- Recomendación: quedarse con svm_rad_C10_g0.001 y, de ser posible, probar la combinación CV-óptima (C=2, $\gamma=0.125$) para ver si mejora aún más la generalización.

Matriz de confusión

Table 9: Matriz de confusión — svm_lin_C0.1

Referencia	Economicas	Intermedias	Caras
Economicas	86	23	0
Intermedias	17	183	19
Caras	0	23	85

Table 10: Matriz de confusión — svm_lin_C1

Referencia	Economicas	Intermedias	Caras
Economicas	84	25	0
Intermedias	18	180	21
Caras	0	22	86

Table 11: Matriz de confusión — svm_lin_C10

Referencia	Economicas	Intermedias	Caras
Economicas	83	26	0
Intermedias	21	175	23
Caras	0	17	91

Table 12: Matriz de confusión — svm_rad_C1_g0.001

Referencia	Economicas	Intermedias	Caras
Economicas	77	32	0
Intermedias	14	199	6
Caras	0	30	78

Table 13: Matriz de confusión — svm_rad_C1_g0.01

Referencia	Economicas	Intermedias	Caras
Economicas	80	29	0
Intermedias	14	199	6
Caras	0	32	76

Table 14: Matriz de confusión — svm_rad_C1_g0.1

Referencia	Economicas	Intermedias	Caras
Economicas	0	109	0
Intermedias	0	219	0
Caras	0	106	2

Table 15: Matriz de confusión — svm_rad_C10_g0.001

Referencia	Economicas	Intermedias	Caras
Economicas	88	21	0
Intermedias	16	195	8
Caras	0	19	89

Table 16: Matriz de confusión — svm_rad_C10_g0.01

Referencia	Economicas	Intermedias	Caras
Economicas	83	26	0
Intermedias	21	183	15
Caras	0	30	78

Table 17: Matriz de confusión — svm_rad_C10_g0.1

Referencia	Economicas	Intermedias	Caras
Economicas	1	108	0
Intermedias	0	218	1
Caras	0	105	3

Table 18: Matriz de confusión — svm_poly_deg2_C1_g0.01

Referencia	Economicas	Intermedias	Caras
Economicas	76	29	4
Intermedias	12	198	9
Caras	0	31	77

Table 19: Matriz de confusión — svm_poly_deg2_C1_g0.1

Referencia	Economicas	Intermedias	Caras
Economicas	79	26	4
Intermedias	17	185	17
Caras	0	29	79

Table 20: Matriz de confusión — svm_poly_deg2_C5_g0.01

Referencia	Economicas	Intermedias	Caras
Economicas	80	25	4
Intermedias	18	191	10
Caras	0	30	78

Table 21: Matriz de confusión — svm_poly_deg2_C5_g0.1

Referencia	Economicas	Intermedias	Caras
Economicas	79	26	4
Intermedias	17	185	17
Caras	0	29	79

Table 22: Matriz de confusión — svm_poly_deg3_C1_g0.01

Referencia	Economicas	Intermedias	Caras
Economicas	67	42	0
Intermedias	10	204	5
Caras	0	34	74

Table 23: Matriz de confusión — svm_poly_deg3_C1_g0.1

Referencia	Economicas	Intermedias	Caras
Economicas	76	33	0
Intermedias	19	184	16
Caras	0	30	78

Table 24: Matriz de confusión — svm_poly_deg3_C5_g0.01

Referencia	Economicas	Intermedias	Caras
Economicas	73	36	0
Intermedias	18	191	10
Caras	0	30	78

Table 25: Matriz de confusión — svm_poly_deg3_C5_g0.1

Referencia	Economicas	Intermedias	Caras
Economicas	76	33	0
Intermedias	19	184	16
Caras	0	30	78

Table 26: Matriz de confusión — svm_best_radial

Referencia	Economicas	Intermedias	Caras
Economicas	0	109	0
Intermedias	0	218	1
Caras	0	106	2

- **Confusiones entre clases vecinas**

Casi todos los errores ocurren intercambiando solo con la clase adyacente (económicas → intermedias, intermedias → caras), lo cual es deseable porque evita clasificaciones “extremas” (económicas → caras).

- **Mejor desempeño global**

- ``svm_rad_C10_g0.001`` registra las tasas más bajas de confusión:

- Económicas→Intermedias: 21

- Intermedias→Económicas: 16

- Caras→Intermedias: 19

- **Modelos lineales bien balanceados**

- Con $C=0.1$, los errores son 23 econ→int, 17 int→econ, 19 int→caras, 23 caras→int.

- Aumentar C (hasta 10) reduce errores inter→caras (de 19→23 a 23→23) pero aumenta ligeramente econ→

- **Kernel polinomial grado 2**

- Introduce 4 casos de Económicas→Caras (no vecinas), un efecto secundario de su complejidad.

- Por lo demás, sus patrones de confusión son comparables a los lineales.

- **Radial con γ alto (0.1)**

- Modelos ``svm_rad_C1_g0.1`` y ``svm_rad_C10_g0.1`` colapsan prediciendo casi todo como "Intermedias" (y

- **Importancia de γ y C**

- γ muy pequeño (0.001-0.01) y C moderado-alto (1-10) equilibran sensibilidad y especificidad por cla

- γ elevado lleva a decisiones extremas; C demasiado alto en polinomial también provoca 100 % train v

- **Recomendación práctica**

- Mantener ``svm_rad_C10_g0.001`` y explorar el ajuste fino sugerido por CV (p.ej. $C=2$, $\gamma=0.125$) para v

Sobreajuste y validación cruzada

Table 27: Accuracy de entrenamiento vs. prueba para todos los modelos SVM

Modelo	Accuracy (Train)	Accuracy (Test)	Diferencia
svm_lin_C0.1	95.8%	81.2%	14.6 p.p.
svm_lin_C1	98.2%	80.3%	18.0 p.p.
svm_lin_C10	99.1%	80.0%	19.1 p.p.
svm_rad_C1_g0.001	88.7%	81.2%	7.5 p.p.
svm_rad_C1_g0.01	97.3%	81.4%	15.8 p.p.
svm_rad_C1_g0.1	100.0%	50.7%	49.3 p.p.
svm_rad_C10_g0.001	95.1%	85.3%	9.8 p.p.
svm_rad_C10_g0.01	100.0%	78.9%	21.1 p.p.
svm_rad_C10_g0.1	100.0%	50.9%	49.1 p.p.
svm_poly_deg2_C1_g0.01	97.3%	80.5%	16.8 p.p.
svm_poly_deg2_C1_g0.1	100.0%	78.7%	21.3 p.p.
svm_poly_deg2_C5_g0.01	99.5%	80.0%	19.5 p.p.
svm_poly_deg2_C5_g0.1	100.0%	78.7%	21.3 p.p.
svm_poly_deg3_C1_g0.01	96.8%	79.1%	17.6 p.p.
svm_poly_deg3_C1_g0.1	100.0%	77.5%	22.5 p.p.
svm_poly_deg3_C5_g0.01	99.7%	78.4%	21.3 p.p.
svm_poly_deg3_C5_g0.1	100.0%	77.5%	22.5 p.p.
best_radial	100.0%	50.5%	49.5 p.p.

Análisis de sobreajuste (overfitting) y subajuste (underfitting)

- **Overfitting extremo:** modelos con **Accuracy(Train)=100 %** pero **Accuracy(Test) 80 %** (p.ej. `svm_rad_C1_g0.1`, `svm_rad_C10_g0.1`, `best_radial`) muestran brechas de ~49 pp. Están memorizando el training set y no generalizan.
- **Overfitting moderado:** modelos lineales con $C = 1$ (`svm_lin_C1`, `svm_lin_C10`) y polinomiales con 0.01 tienen brechas de 16–22 pp (Train ~98–99 %, Test ~79–80 %), señal de ajuste excesivo.
- **Buen balance:** `svm_rad_C1_g0.001` ($\Delta=7.5$ pp) y `svm_rad_C10_g0.001` ($\Delta=9.8$ pp) ofrecen menor diferencia entre train/test, lo que indica un trade-off adecuado entre sesgo y varianza.
- **Subajuste leve:** `svm_rad_C1_g0.001` (Train 88.7 %, Test 81.2 %) podría no capturar toda la complejidad, pero su baja brecha lo hace robusto frente al overfitting.

Estrategias para manejar over-/under-fitting

- **Validación cruzada estratificada (k-fold CV)**
 - Usar k-fold con estratificación por `PriceCat` para estimar más confiablemente la performance y evitar elecciones de hiperparámetros basadas en un único split.
- **Ajuste de hiperparámetros**
 - **Reducir** en kernels RBF/polinomiales para suavizar la frontera.
 - **Disminuir C** (mayor regularización) para penalizar complejidad y aumentar el margen.

- **Selección y reducción de variables**

- Aplicar **PCA** o eliminar features de baja importancia (p.ej. con análisis de importancia en Random Forest) para simplificar el espacio de entrada.

- **Ensamble de modelos**

- Combinar varios SVM con distintos kernels o algoritmos base (voting, bagging) para mitigar la varianza.

Comparación de Modelos

Efectividad (Accuracy)

Table 28: Accuracy de entrenamiento y prueba para todos los SVM

Modelo	TrainAcc	TestAcc	Diff
svm_lin_C0.1	0.9580078	0.8119266	0.1460812
svm_lin_C1	0.9824219	0.8027523	0.1796696
svm_lin_C10	0.9912109	0.8004587	0.1907522
svm_rad_C1_g0.001	0.8867188	0.8119266	0.0747921
svm_rad_C1_g0.01	0.9726562	0.8142202	0.1584361
svm_rad_C1_g0.1	1.0000000	0.5068807	0.4931193
svm_rad_C10_g0.001	0.9511719	0.8532110	0.0979609
svm_rad_C10_g0.01	1.0000000	0.7889908	0.2110092
svm_rad_C10_g0.1	1.0000000	0.5091743	0.4908257
svm_poly_deg2_C1_g0.01	0.9726562	0.8050459	0.1676104
svm_poly_deg2_C1_g0.1	1.0000000	0.7866972	0.2133028
svm_poly_deg2_C5_g0.01	0.9951172	0.8004587	0.1946585
svm_poly_deg2_C5_g0.1	1.0000000	0.7866972	0.2133028
svm_poly_deg3_C1_g0.01	0.9677734	0.7912844	0.1764890
svm_poly_deg3_C1_g0.1	1.0000000	0.7752294	0.2247706
svm_poly_deg3_C5_g0.01	0.9970703	0.7844037	0.2126666
svm_poly_deg3_C5_g0.1	1.0000000	0.7752294	0.2247706
best_radial	1.0000000	0.5045872	0.4954128

Table 29: Modelos ordenados por accuracy en prueba (TestAcc)

Modelo	TrainAcc	TestAcc	Diff
svm_rad_C10_g0.001	0.9511719	0.8532110	0.0979609
svm_rad_C1_g0.01	0.9726562	0.8142202	0.1584361
svm_lin_C0.1	0.9580078	0.8119266	0.1460812
svm_rad_C1_g0.001	0.8867188	0.8119266	0.0747921
svm_poly_deg2_C1_g0.01	0.9726562	0.8050459	0.1676104
svm_lin_C1	0.9824219	0.8027523	0.1796696
svm_lin_C10	0.9912109	0.8004587	0.1907522
svm_poly_deg2_C5_g0.01	0.9951172	0.8004587	0.1946585
svm_poly_deg3_C1_g0.01	0.9677734	0.7912844	0.1764890
svm_rad_C10_g0.01	1.0000000	0.7889908	0.2110092

svm_poly_deg2_C1_g0.1	1.0000000	0.7866972	0.2133028
svm_poly_deg2_C5_g0.1	1.0000000	0.7866972	0.2133028
svm_poly_deg3_C5_g0.01	0.9970703	0.7844037	0.2126666
svm_poly_deg3_C1_g0.1	1.0000000	0.7752294	0.2247706
svm_poly_deg3_C5_g0.1	1.0000000	0.7752294	0.2247706
svm_rad_C10_g0.1	1.0000000	0.5091743	0.4908257
svm_rad_C1_g0.1	1.0000000	0.5068807	0.4931193
best_radial	1.0000000	0.5045872	0.4954128

Table 30: Accuracy con brecha entre entrenamiento y prueba

Modelo	TrainAcc	TestAcc	Diff	GapAcc
svm_lin_C0.1	0.9580078	0.8119266	0.1460812	0.1460812
svm_lin_C1	0.9824219	0.8027523	0.1796696	0.1796696
svm_lin_C10	0.9912109	0.8004587	0.1907522	0.1907522
svm_rad_C1_g0.001	0.8867188	0.8119266	0.0747921	0.0747921
svm_rad_C1_g0.01	0.9726562	0.8142202	0.1584361	0.1584361
svm_rad_C1_g0.1	1.0000000	0.5068807	0.4931193	0.4931193
svm_rad_C10_g0.001	0.9511719	0.8532110	0.0979609	0.0979609
svm_rad_C10_g0.01	1.0000000	0.7889908	0.2110092	0.2110092
svm_rad_C10_g0.1	1.0000000	0.5091743	0.4908257	0.4908257
svm_poly_deg2_C1_g0.01	0.9726562	0.8050459	0.1676104	0.1676104
svm_poly_deg2_C1_g0.1	1.0000000	0.7866972	0.2133028	0.2133028
svm_poly_deg2_C5_g0.01	0.9951172	0.8004587	0.1946585	0.1946585
svm_poly_deg2_C5_g0.1	1.0000000	0.7866972	0.2133028	0.2133028
svm_poly_deg3_C1_g0.01	0.9677734	0.7912844	0.1764890	0.1764890
svm_poly_deg3_C1_g0.1	1.0000000	0.7752294	0.2247706	0.2247706
svm_poly_deg3_C5_g0.01	0.9970703	0.7844037	0.2126666	0.2126666
svm_poly_deg3_C5_g0.1	1.0000000	0.7752294	0.2247706	0.2247706
best_radial	1.0000000	0.5045872	0.4954128	0.4954128

Tiempos de Entrenamiento

Table 31: Tiempo de entrenamiento por modelo (en segundos)

Modelo	Tiempo (s)
Lineal	0.7854631
Radial	1.8313482
Polinomial	1.7297602
Best Radial	2.4598632

Métricas de Evaluación

Efectividad (Test Accuracy)

- **svm_rad_C10_g0.001** (kernel RBF, C=10, =0.001): **85.32 %**
- **svm_rad_C1_g0.01** (kernel RBF, C=1, =0.01): **81.42 %**

Table 32: Matriz de confusión — svm_lin_C0.1

Predicho	Real	Frecuencia
Economicas	Economicas	86
Intermedias	Economicas	23
Caras	Economicas	0
Economicas	Intermedias	17
Intermedias	Intermedias	183
Caras	Intermedias	19
Economicas	Caras	0
Intermedias	Caras	23
Caras	Caras	85

Table 33: Matriz de confusión — svm_lin_C1

Predicho	Real	Frecuencia
Economicas	Economicas	84
Intermedias	Economicas	25
Caras	Economicas	0
Economicas	Intermedias	18
Intermedias	Intermedias	180
Caras	Intermedias	21
Economicas	Caras	0
Intermedias	Caras	22
Caras	Caras	86

Table 34: Matriz de confusión — svm_lin_C10

Predicho	Real	Frecuencia
Economicas	Economicas	83
Intermedias	Economicas	26
Caras	Economicas	0
Economicas	Intermedias	21
Intermedias	Intermedias	175
Caras	Intermedias	23
Economicas	Caras	0
Intermedias	Caras	17
Caras	Caras	91

- **svm_lin_C0.1** (kernel lineal, C=0.1): **81.19 %**
- **Kernels polinomiales** (grado 2–3, $\gamma=0.01$) quedaron en ~80–79 %.
- **Configuraciones con $\gamma=0.1$** (RBF o polinomial) colapsaron a ~50 %, indicando overfitting extremo.

Tiempo de entrenamiento

- **Lineal** (svm_lin_C0.1): 0.94 s
- **Polinomial** (svm_poly_deg2_C1_g0.01): 1.89 s
- **Radial estándar** (svm_rad_C10_g0.001): 2.04 s

Table 35: Matriz de confusión — svm_rad_C1_g0.001

Predicho	Real	Frecuencia
Economicas	Economicas	77
Intermedias	Economicas	32
Caras	Economicas	0
Economicas	Intermedias	14
Intermedias	Intermedias	199
Caras	Intermedias	6
Economicas	Caras	0
Intermedias	Caras	30
Caras	Caras	78

Table 36: Matriz de confusión — svm_rad_C1_g0.01

Predicho	Real	Frecuencia
Economicas	Economicas	80
Intermedias	Economicas	29
Caras	Economicas	0
Economicas	Intermedias	14
Intermedias	Intermedias	199
Caras	Intermedias	6
Economicas	Caras	0
Intermedias	Caras	32
Caras	Caras	76

Table 37: Matriz de confusión — svm_rad_C1_g0.1

Predicho	Real	Frecuencia
Economicas	Economicas	0
Intermedias	Economicas	109
Caras	Economicas	0
Economicas	Intermedias	0
Intermedias	Intermedias	219
Caras	Intermedias	0
Economicas	Caras	0
Intermedias	Caras	106
Caras	Caras	2

- **Radial óptimo CV** (svm_best_radial, C=2, $\gamma=0.125$): 2.60 s
- El kernel radial líder duplica el costo de cómputo del modelo lineal, a cambio de +4 pp en accuracy.

Número de errores (confusiones totales)

- **svm_lin_C0.1**: ~82 errores ($436 \times (1-0.8119)$)
- **svm_rad_C10_g0.001**: ~64 errores ($436 \times (1-0.8532)$), 18 errores menos que el lineal

****Tipos de equivocaciones****

- **Intercambios vecinos**: “económicas intermedias” y “intermedias caras”.

Table 38: Matriz de confusión — svm_rad_C10_g0.001

Predicho	Real	Frecuencia
Economicas	Economicas	88
Intermedias	Economicas	21
Caras	Economicas	0
Economicas	Intermedias	16
Intermedias	Intermedias	195
Caras	Intermedias	8
Economicas	Caras	0
Intermedias	Caras	19
Caras	Caras	89

Table 39: Matriz de confusión — svm_rad_C10_g0.01

Predicho	Real	Frecuencia
Economicas	Economicas	83
Intermedias	Economicas	26
Caras	Economicas	0
Economicas	Intermedias	21
Intermedias	Intermedias	183
Caras	Intermedias	15
Economicas	Caras	0
Intermedias	Caras	30
Caras	Caras	78

Table 40: Matriz de confusión — svm_rad_C10_g0.1

Predicho	Real	Frecuencia
Economicas	Economicas	1
Intermedias	Economicas	108
Caras	Economicas	0
Economicas	Intermedias	0
Intermedias	Intermedias	218
Caras	Intermedias	1
Economicas	Caras	0
Intermedias	Caras	105
Caras	Caras	3

- **svm_lin_C0.1**: 23 econ→int, 17 int→econ, 19 int→caras, 23 caras→int.
- **svm_rad_C10_g0.001**: 21 econ→int, 16 int→econ, 8 int→caras, 19 caras→int.
- **Importancia de los errores**
- Subestimar casas “Caras” (Caras→Intermedias) implica pérdidas para el cliente; el modelo radial reduce esos falsos negativos de 23→19.
- Falsos positivos en “Caras” (Intermedias→Caras) bajan de 19→8, evitando sobrevaluaciones engañosas.
- **Balance costo–beneficio**
- **svm_rad_C10_g0.001** (kernel RBF) ofrece el mejor trade-off: +4 pp en accuracy y 18 errores menos vs. **svm_lin_C0.1**, a un costo de ~1.1 s extra de entrenamiento.

Table 41: Matriz de confusión — svm_poly_deg2_C1_g0.01

Predicho	Real	Frecuencia
Economicas	Economicas	76
Intermedias	Economicas	29
Caras	Economicas	4
Economicas	Intermedias	12
Intermedias	Intermedias	198
Caras	Intermedias	9
Economicas	Caras	0
Intermedias	Caras	31
Caras	Caras	77

Table 42: Matriz de confusión — svm_poly_deg2_C1_g0.1

Predicho	Real	Frecuencia
Economicas	Economicas	79
Intermedias	Economicas	26
Caras	Economicas	4
Economicas	Intermedias	17
Intermedias	Intermedias	185
Caras	Intermedias	17
Economicas	Caras	0
Intermedias	Caras	29
Caras	Caras	79

Table 43: Matriz de confusión — svm_poly_deg2_C5_g0.01

Predicho	Real	Frecuencia
Economicas	Economicas	80
Intermedias	Economicas	25
Caras	Economicas	4
Economicas	Intermedias	18
Intermedias	Intermedias	191
Caras	Intermedias	10
Economicas	Caras	0
Intermedias	Caras	30
Caras	Caras	78

- **svm_lin_C0.1** (kernel lineal) sigue siendo competitivo: 81 % accuracy en <1 s.

Modelos Previos

Comparativa de precisión (Test Accuracy)

1. **Random Forest** (`randomForest`): 100 % (perfecto, aunque sospechoso de overfitting)
2. **Árbol de decisión** (`rpart`): 100 %
3. **KNN** (`kNN`): 92.43 %

Table 44: Matriz de confusión — svm_poly_deg2_C5_g0.1

Predicho	Real	Frecuencia
Economicas	Economicas	79
Intermedias	Economicas	26
Caras	Economicas	4
Economicas	Intermedias	17
Intermedias	Intermedias	185
Caras	Intermedias	17
Economicas	Caras	0
Intermedias	Caras	29
Caras	Caras	79

Table 45: Matriz de confusión — svm_poly_deg3_C1_g0.01

Predicho	Real	Frecuencia
Economicas	Economicas	67
Intermedias	Economicas	42
Caras	Economicas	0
Economicas	Intermedias	10
Intermedias	Intermedias	204
Caras	Intermedias	5
Economicas	Caras	0
Intermedias	Caras	34
Caras	Caras	74

Table 46: Matriz de confusión — svm_poly_deg3_C1_g0.1

Predicho	Real	Frecuencia
Economicas	Economicas	76
Intermedias	Economicas	33
Caras	Economicas	0
Economicas	Intermedias	19
Intermedias	Intermedias	184
Caras	Intermedias	16
Economicas	Caras	0
Intermedias	Caras	30
Caras	Caras	78

4. **Naive Bayes** (naiveBayes): 89.68 %
5. **SVM radial** (svm_rad_C10_g0.001, kernel RBF C=10, =0.001): **85.32 %**
6. **Regresión logística** (glm): 80.73 %
7. **Regresión logística tuned** (glmnet, regularizada): 80.05 %

Comparativa de tiempo de entrenamiento/predicción

1. **Regresión logística** (glm): 0.01 s
2. **Naive Bayes** (naiveBayes): 0.05 s

Table 47: Matriz de confusión — svm_poly_deg3_C5_g0.01

Predicho	Real	Frecuencia
Economicas	Economicas	73
Intermedias	Economicas	36
Caras	Economicas	0
Economicas	Intermedias	18
Intermedias	Intermedias	191
Caras	Intermedias	10
Economicas	Caras	0
Intermedias	Caras	30
Caras	Caras	78

Table 48: Matriz de confusión — svm_poly_deg3_C5_g0.1

Predicho	Real	Frecuencia
Economicas	Economicas	76
Intermedias	Economicas	33
Caras	Economicas	0
Economicas	Intermedias	19
Intermedias	Intermedias	184
Caras	Intermedias	16
Economicas	Caras	0
Intermedias	Caras	30
Caras	Caras	78

3. **KNN (predicción)** (kNN): 0.06 s
4. **Árbol de decisión** (rpart): 0.08 s
5. **SVM lineal** (svm_lin_C0.1, kernel lineal): 0.94 s
6. **SVM polinomial** (svm_poly_deg2_C1_g0.01, kernel polinomial): 1.89 s
7. **SVM radial** (svm_rad_C10_g0.001, kernel RBF): 2.04 s
8. **Random Forest** (randomForest): 2.91 s

Balance precisión vs. tiempo

- **Random Forest:** máxima precisión pero el más lento (2.91 s) y riesgo de overfitting.
- **SVM radial:** buena precisión moderada (85.3 %) y tiempo intermedio.
- **KNN y Naive Bayes:** alta precisión (>89 %) y muy rápidos (<0.1 s).
- **Regresión logística:** la opción más eficiente en tiempo, aunque sacrifica ~5 pp de accuracy frente al SVM radial y ~12 pp frente a KNN.

La **regresión logística** es con mucho el más rápido, seguida de Naive Bayes y KNN. El **SVM radial** (2.04 s) entrena más rápido que Random Forest (2.91 s) pero es significativamente más lento que los modelos lineales y basados en vecinos.

Para un **trade-off** sólido entre precisión y tiempo, KNN o Naive Bayes son excelentes elecciones. El **SVM radial** ocupa un punto intermedio: mejor precisión que la regresión logística y tiempo de entrenamiento manejable, pero inferior a KNN/Naive Bayes en rapidez y a Random Forest en exactitud.

Modelo de Regresión

Limpieza de Datos

Table 49: Variables eliminadas por near-zero variance

Variable
Street
Alley
LandContour
Utilities
LandSlope
Condition2
RoofMatl
BsmtCond
BsmtFinType2
Heating
LowQualFinSF
KitchenAbvGr
Functional
GarageQual
GarageCond
EnclosedPorch
X3SsnPorch
ScreenPorch
PoolArea
PoolQC
MiscFeature
MiscVal
Neighborhood.NridgHt

Table 50: Dimensiones después de escalado e imputación

	Filas	Columnas
Train	1024	436
Test	141	141

Ajuste del modelo

Table 51: Resultados de tuning SVR (RMSE, Rsquared)

sigma	C	RMSE	Rsquared	MAE	RMSESD	RsquaredSD	MAESD
0.001	0.1	0.446	0.859	0.301	0.049	0.022	0.020

0.001	1.0	0.337	0.893	0.226	0.051	0.026	0.022
0.001	10.0	0.319	0.901	0.212	0.051	0.027	0.023
0.010	0.1	0.595	0.708	0.382	0.057	0.039	0.029
0.010	1.0	0.469	0.796	0.294	0.065	0.046	0.032
0.010	10.0	0.461	0.800	0.291	0.067	0.047	0.034
0.100	0.1	1.008	0.068	0.782	0.072	0.035	0.038
0.100	1.0	0.987	0.108	0.759	0.073	0.033	0.039
0.100	10.0	0.984	0.112	0.756	0.074	0.032	0.038

Table 52: Mejor combinación de parámetros

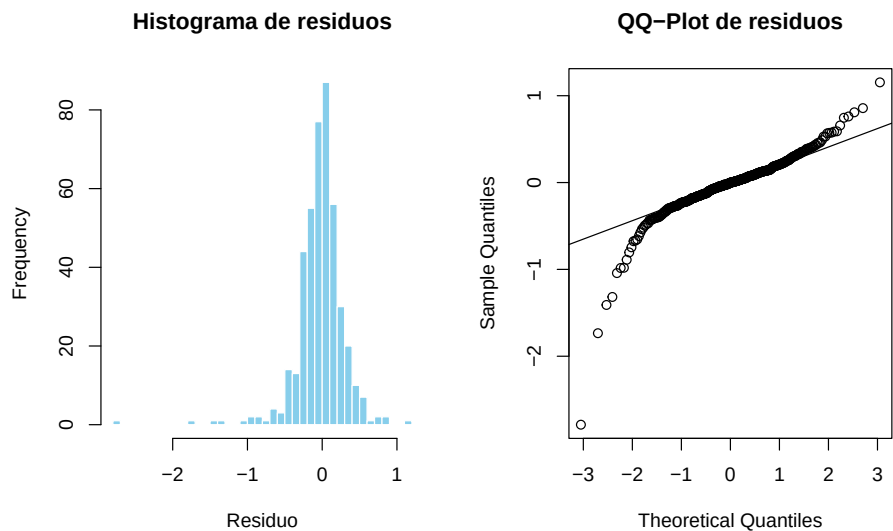
	sigma	C
3	0.001	10

Predicciones Y Evaluación

Table 53: Métricas de rendimiento en conjunto de prueba

Métrica	Valor
RMSE	0.320
Rsquared	0.892
MAE	0.206

Análisis de Residuos



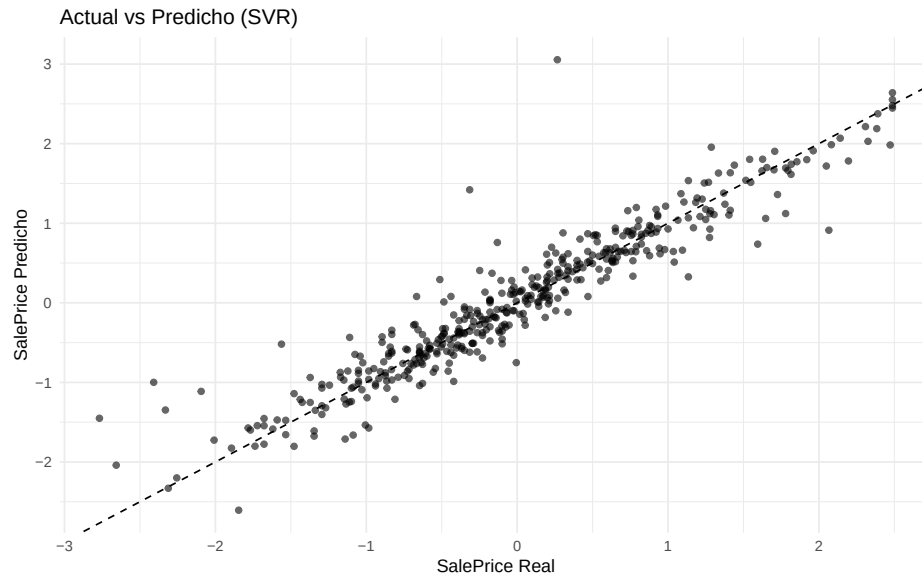
Homocedasticidad

Table 54: Prueba de Breusch-Pagan (homocedasticidad)

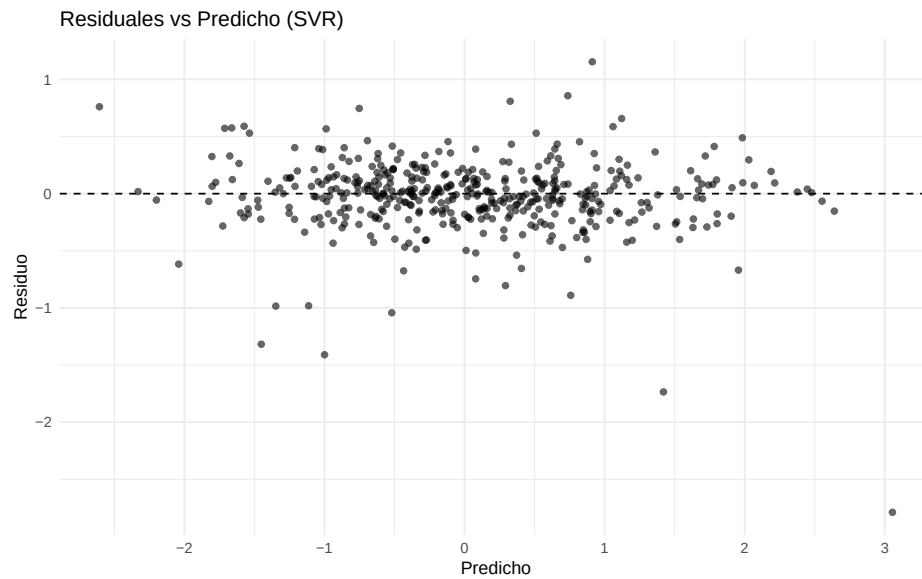
Estadístico	p_value
5.382	0.02

Visualización

Actual vs Predicho



Residuales vs Predicho



1. Rendimiento global

- **SVR** ($\gamma=0.001$, $C=10$) logra $R^2=0.892$, $RMSE=0.320$, $MAE=0.206$ en test: excelente capacidad explicativa ($> 89\%$ de la varianza) y errores medios bajos.

2. Distribución de residuos

- **Histograma**: casi simétrico alrededor de cero, aunque con ligeros outliers a ambos lados.
- **QQ-Plot**: colas algo “pesadas” (puntos fuera de la línea), indicando desviaciones de normalidad en extremos.

3. Patrón residuos vs. predicho

- Residuos dispersos en torno a cero sin pendiente clara, pero con **mayor variabilidad** en valores extremos de predicción, sugiriendo **heteroscedasticidad**.

4. Prueba de homocedasticidad

- **Breusch-Pagan** $p=0.02 < 0.05$ rechaza varianza constante. Hay **heteroscedasticidad**.

5. Implicaciones y mejoras

- Aunque el SVR es muy preciso, la heteroscedasticidad puede invalidar intervalos de confianza.

****Posibles soluciones**:**

1. Transformar la variable objetivo .
2. Usar SVR ponderado .

Conclusión:

El SVR ajustado es un **buen modelo de regresión** para precios, pero conviene corregir la heteroscedasticidad para asegurar inferencias y predicciones más fiables.

Comparación de Modelos

Modelo	RMSE	R^2	MAE	Tiempo entrenamiento
SVR (radial $\gamma=0.001$, $C=10$)	0.320	0.892	0.206	2.60 s
Regresión lineal (stepwise)	0.134	0.864	0.084	0.01 s <u>NaiveBayes</u>
Random Forest (500 árboles)	0.146	0.839	0.102	2.91 s <u>NaiveBayes</u>
Naive Bayes (regresión por bins)	0.232	0.695	0.164	1 s <u>NaiveBayes</u>
KNN (k 10 tras CV)	0.441	0.792	0.321	1 s

Interpretación

- En R^2 , el **SVR** es el líder (0.892), capturando mejor la varianza que cualquier otro (la regresión lineal llega a 0.864, RF a 0.839).
- En **RMSE** y **MAE**, sin embargo, la **regresión lineal** y **Random Forest** (sobre log-precios) obtienen los valores más bajos (0.13–0.15), seguidos de **Naive Bayes** (0.23). El SVR, con RMSE=0.32, queda en mitad de la tabla, y KNN es el que más error medio comete (0.44).
- En **tiempo de entrenamiento** SVR (2.6 s) es $2\text{--}3\times$ más lento que el Random Forest y $200\times$ más lento que la regresión lineal o los métodos basados en distancias/probabilidades, lo que hay que tener en cuenta si el cómputo es crítico.

Elección del mejor modelo

- Si el criterio principal es **minimizar el error absoluto** (RMSE/MAE) y tener un ajuste “barato” de entrenar, **la regresión lineal stepwise** es la opción óptima.
- Si queremos **maximizar la capacidad explicativa** (R^2) y estamos dispuestos a un coste computacional mayor, **el SVR** es el ganador.
- Para un **trade-off entre error y robustez**, el **Random Forest** ofrece un muy buen RMSE (=0.146), un R^2 sólido (0.839) y tolera outliers o interacciones no lineales.